Assignment #5 — Strategy Pattern: City Route Planner

Student: Begaliev Bakdaulet
Group: SE-2526
Course: Software Design Patterns (ShP-2216)
Academic year: 2026–2027
Language: Java 17
GitHub: https://github.com/bakdauletbegaliev/strategy

1. Introduction

My project is a small route planner. The user has one trip with a start location, a destination, and a known distance. The program can estimate travel time and price for three transport types: walking, bicycle, or car.

These are three interchangeable algorithms. Walking uses 5 km/h and no cost; bicycle uses 14 km/h plus 2 minutes for parking; car uses 38 km/h plus 8 minutes for traffic/parking, with a simple estimated cost of `distance × 85 + 250` KZT. These are educational estimates, not real navigation or live traffic information.

The **Strategy interface** is `RouteStrategy`. Its `calculate(Route)` method is implemented by the three concrete strategies. `Navigator` is the **Context**. The `Main` class is the **Client** and can select a strategy while the program is running.

2. UML Class Diagram

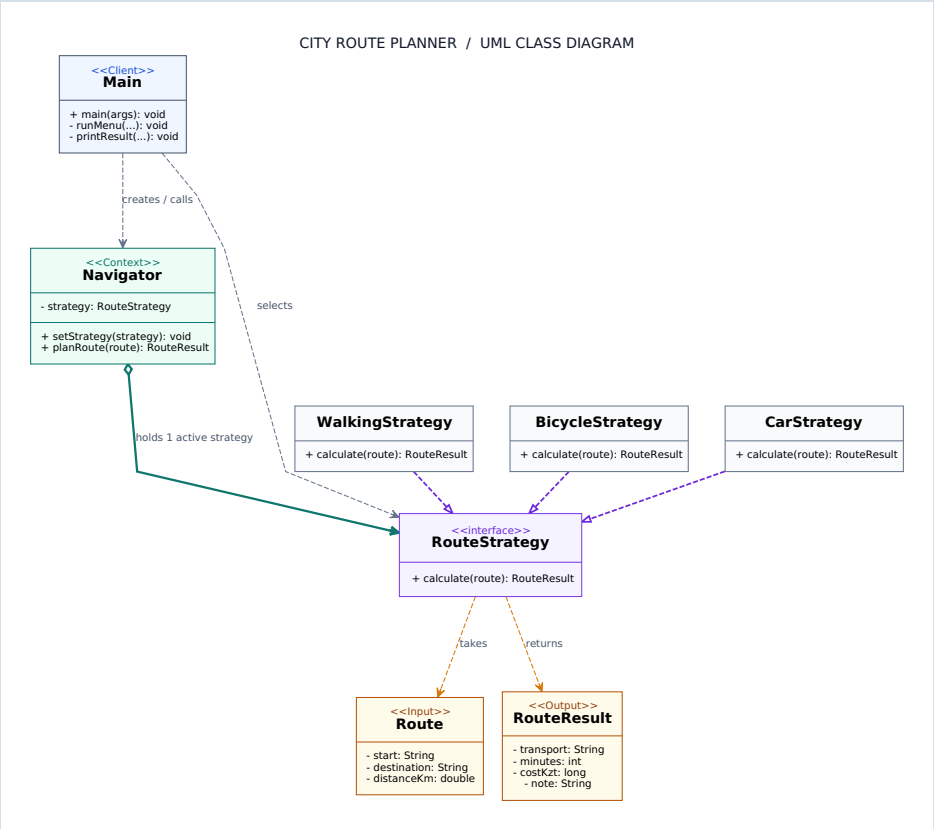

Figure 1. UML class diagram of City Route Planner. It shows the Client, Context, Strategy interface, three concrete strategies, and route data classes.

Diagram files: report PNG · full-size UML · PlantUML source.

Different UML arrows have different meanings:

- **Hollow diamond + solid line:** `Navigator` aggregates (keeps a reference to) the selected `RouteStrategy`; the strategy object can exist independently of the navigator.
- **Hollow triangle + dashed line:** `WalkingStrategy`, `BicycleStrategy`, and `CarStrategy` implement `RouteStrategy` (interface realization).
- **Dashed dependency arrow:** `Main` creates/uses `Navigator` and selects a strategy; `RouteStrategy` uses `Route` and returns `RouteResult`.

Runtime switching happens here:

```
navigator.setStrategy(new BicycleStrategy());
RouteResult result = navigator.planRoute(route);
```

There is no `if` or `switch` on transport types inside `Navigator`:

```
public RouteResult planRoute(Route route) {
    Objects.requireNonNull(route, "Route cannot be null");
    return strategy.calculate(route);
}
```

3. Clean Code Principles

The **before** snippets below are simplified examples of designs I avoided; they are not excerpts from an earlier Git version. The **after** snippets describe the actual solution.

3.1 Replace conditional algorithms with polymorphism

Before:

```
if (transport.equals("car")) {
    return carCalculation(route);
} else if (transport.equals("walking")) {
    return walkingCalculation(route);
}
```

After:

```
return strategy.calculate(route);
```

Reason: The Context delegates to whichever strategy object is selected, without checking its concrete type.

3.2 Use meaningful class and method names

Before:

```
class S {
    int calc(double d) { return 0; }
}
```

After:

```
public class WalkingStrategy implements RouteStrategy {
    public RouteResult calculate(Route route) { ... }
}
```

Reason: The class name explains the transport type, and `calculate` tells us what the method does.

3.3 Follow the Single Responsibility Principle

Before:

```
class Navigator {
    // calculates walking, bicycle and car prices and times
}
```

After:

```
class CarStrategy implements RouteStrategy { ... }
class BicycleStrategy implements RouteStrategy { ... }
class WalkingStrategy implements RouteStrategy { ... }
```

Reason: Each algorithm belongs to one class. `Navigator` only stores the currently selected strategy and delegates the calculation.

3.4 Keep the Context independent of concrete strategies

Before:

```
class Navigator {
    private CarStrategy strategy = new CarStrategy();
}
```

After:

```
private RouteStrategy strategy;

public void setStrategy(RouteStrategy strategy) {
    this.strategy = Objects.requireNonNull(strategy);
}
```

Reason: The Context depends on the interface rather than a particular transport class.

3.5 Make it easy to extend without modifying old classes

Before:

```
if (transport.equals("scooter")) {
    // add another branch in Navigator
}
```

After:

```
class ScooterStrategy implements RouteStrategy {
    public RouteResult calculate(Route route) {
        return new RouteResult("Scooter", 25, 300, "Scooter trip");
    }
}
```

Reason: A new transport method only needs a new class implementing `RouteStrategy`. The existing `Navigator`, `Route`, and strategy classes do not need changes. The test also shows that a new strategy can be injected dynamically.

4. Strategy vs. State

Strategy fits because the **Client chooses** how to perform one calculation: walking, bicycle, or car. The Context does not decide when a transport mode should change; the user selects it. The **State** pattern would be more appropriate if the object's behavior changed

automatically because of its internal state and allowed state transitions (for example, an order moving from `Created` to `Paid` to `Shipped`). Both patterns use composition, but they solve different problems.

5. Conclusion

The Strategy pattern replaces a large transport `if/else` chain with a small interface and separate classes. `Navigator` stays simple, and new transport algorithms can be added without rewriting it. The trade-off is having more classes, but those classes are small and easy to understand and test. The demonstration uses one `Navigator` and swaps algorithms while the program is running.

6. GitHub Repository

https://github.com/bakdauletbegaliev/strategy

References

1. Astana IT University. *Assignment #5 — Strategy Pattern* (2026–2027), provided course brief.
2. Freeman, E., & Robson, E. (2021). *Head First Design Patterns*, 2nd ed., Chapter 1.
3. Martin, R. C. (2008). *Clean Code*, Chapter 10.
4. Refactoring.Guru. *Strategy*. https://refactoring.guru/design-patterns/strategy